

BMI 500 Profiling Exercise Report

Heng Sun

Slurm batch scripts

pbmc3k.sh

```
#!/bin/bash -l
#SBATCH -J pbmc3k_profile
#SBATCH -p overflow
#SBATCH --mem=4G
#SBATCH --cpus-per-task=1
#SBATCH --time=1:00:00
#SBATCH --output=./slurm_outputs/pbmc3k_%j.out

source ~/miniconda3/etc/profile.d/conda.sh
conda activate bmi500
cd ~/bmi500

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc3k \
  --out-dir data \
  --num-threads 1
```

pbmc6k.sh

```
#!/bin/bash -l
#SBATCH -J pbmc6k_profile
#SBATCH -p overflow
#SBATCH --mem=4G
#SBATCH --cpus-per-task=1
#SBATCH --time=1:00:00
#SBATCH --output=./slurm_outputs/pbmc6k_%j.out

source ~/miniconda3/etc/profile.d/conda.sh
conda activate bmi500
cd ~/bmi500

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc6k \
  --out-dir data \
  --num-threads 1
```

pbmc10k.sh

```
#!/bin/bash -l
#SBATCH -J pbmc10k_profile
#SBATCH -p overflow
#SBATCH --mem=4G
#SBATCH --cpus-per-task=1
#SBATCH --time=1:00:00
#SBATCH --output=./slurm_outputs/pbmc10k_%j.out

source ~/miniconda3/etc/profile.d/conda.sh
conda activate bmi500
cd ~/bmi500

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc10k \
  --out-dir data \
  --num-threads 1
```

Console output captures

PBMC3K

```
[PROFILE] Imports: 2.381 seconds
[PROFILE] Settings: 8.507 seconds
[PROFILE] Argument parsing: 0.001 seconds
[PROFILE] Data loading: 0.107 seconds
[PROFILE] Basic filtering: 0.041 seconds
[PROFILE] Normalization: 0.009 seconds
[PROFILE] Highly variable genes: 0.825 seconds
[PROFILE] Scaling: 0.030 seconds
[PROFILE] Report adata section: 0.000 seconds
[PROFILE] PCA: 0.179 seconds
[PROFILE] Neighbors: 3.106 seconds
[PROFILE] PAGA/commented section: 0.000 seconds
[PROFILE] Louvain clustering: 0.082 seconds
[PROFILE] UMAP: 5.296 seconds
[PROFILE] Write output: 0.105 seconds
[PROFILE] Rank genes: 1.332 seconds
Percent of CPU this job got: 96%
Elapsed (wall clock) time: 0:22.64
Maximum resident set size (kbytes): 935276
Exit status: 0
```

PBMC6K

```
[PROFILE] Imports: 1.792 seconds
[PROFILE] Settings: 4.683 seconds
[PROFILE] Argument parsing: 0.001 seconds
[PROFILE] Data loading: 0.663 seconds
[PROFILE] Basic filtering: 0.046 seconds
[PROFILE] Normalization: 0.013 seconds
[PROFILE] Highly variable genes: 0.023 seconds
[PROFILE] Scaling: 0.058 seconds
[PROFILE] Report adata section: 0.000 seconds
[PROFILE] PCA: 0.347 seconds
[PROFILE] Neighbors: 3.287 seconds
[PROFILE] PAGA/commented section: 0.000 seconds
[PROFILE] Louvain clustering: 0.242 seconds
[PROFILE] UMAP: 10.233 seconds
[PROFILE] Write output: 0.164 seconds
[PROFILE] Rank genes: 2.772 seconds
Percent of CPU this job got: 101%
Elapsed (wall clock) time: 0:25.07
Maximum resident set size (kbytes): 953356
Exit status: 0
```

Console output captures (continued)

PBMC10K

```
[PROFILE] Imports: 3.355 seconds
[PROFILE] Settings: 6.630 seconds
[PROFILE] Argument parsing: 0.001 seconds
[PROFILE] Data loading: 9.973 seconds
[PROFILE] Basic filtering: 2.374 seconds
[PROFILE] Normalization: 0.606 seconds
[PROFILE] Highly variable genes: 1.026 seconds
[PROFILE] Scaling: 0.744 seconds
[PROFILE] Report adata section: 0.000 seconds
[PROFILE] PCA: 1.743 seconds
[PROFILE] Neighbors: 30.527 seconds
[PROFILE] PAGA/commented section: 0.000 seconds
[PROFILE] Louvain clustering: 0.924 seconds
[PROFILE] UMAP: 14.139 seconds
[PROFILE] Write output: 0.948 seconds
[PROFILE] Rank genes: 21.464 seconds
Percent of CPU this job got: 98%
Elapsed (wall clock) time: 1:35.53
Maximum resident set size (kbytes): 1886536
Exit status: 0
```

Coarse-grain profiling results

Dataset	Wall time (s)	CPU	Max RSS (KB)	Max RSS (GiB)
PBMC3K	22.64	96%	935,276	0.89
PBMC6K	25.07	101%	953,356	0.91
PBMC10K	95.53	98%	1,886,536	1.80

PBMC3K and PBMC6K had similar total runtimes and memory use. PBMC10K took 95.53 seconds and used about 1.80 GiB of memory. CPU usage stayed close to one core.

Instrumented profiling results

Code section	3K (s)	6K (s)	10K (s)
Imports	2.381	1.792	3.355
Settings	8.507	4.683	6.630
Argument parsing	0.001	0.001	0.001
Data loading	0.107	0.663	9.973
Basic filtering	0.041	0.046	2.374
Normalization	0.009	0.013	0.606
Highly variable genes	0.825	0.023	1.026
Scaling	0.030	0.058	0.744
Report adata section	0.000	0.000	0.000
PCA	0.179	0.347	1.743
Neighbors	3.106	3.287	30.527
PAGA/commented section	0.000	0.000	0.000
Louvain clustering	0.082	0.242	0.924
UMAP	5.296	10.233	14.139
Write output	0.105	0.164	0.948
Rank genes	1.332	2.772	21.464

Instrumented profiling results (continued)

Instrumented runtime by code section



Figure 1. Instrumented runtime for all code sections.

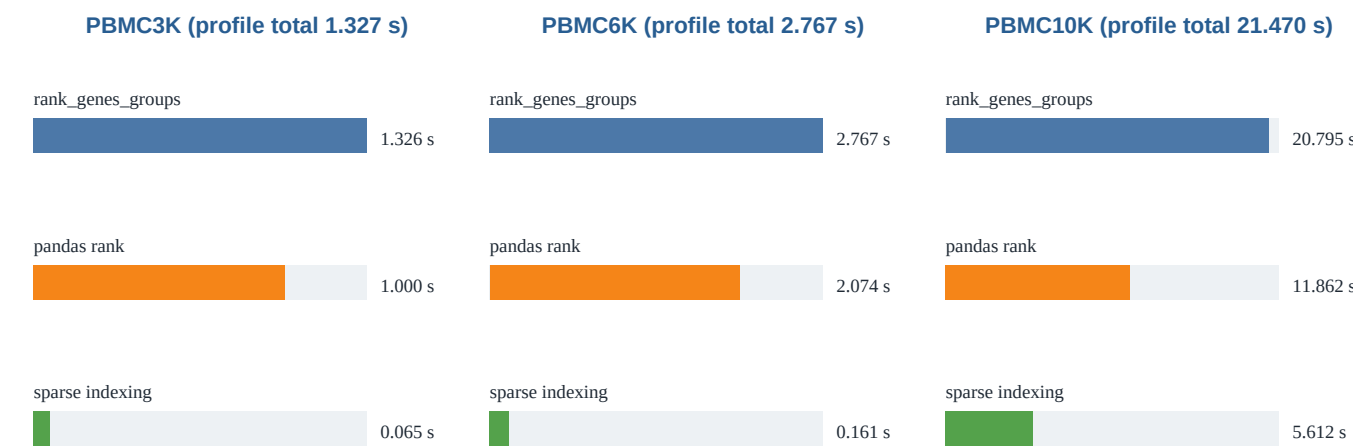
UMAP was the longest section for PBMC3K (5.296 s) and PBMC6K (10.233 s). For PBMC10K, Neighbors was the largest section (30.527 s), followed by Rank genes (21.464 s) and UMAP (14.139 s).

The PBMC3K loading result used an existing cache, while the 6K and 10K runs created a cache file, so the loading times are not directly comparable.

cProfile results for rank_genes_groups

Dataset	Profile total	rank_genes_groups	pandas rank	Sparse indexing
PBMC3K	1.327	1.326	1.000	0.065
PBMC6K	2.767	2.767	2.074	0.161
PBMC10K	21.470	20.795	11.862	5.612

cProfile summary for rank_genes_groups



Bars are scaled within each dataset; values show cumulative time.

Figure 2. Main cumulative-time components of rank_genes_groups for each dataset.

Ranking was the main cost in all three runs. For PBMC10K, sparse indexing also became a noticeable part of the runtime.

Estimated runtimes for 20K cells

I fit a simple linear trend to the 3K, 6K, and 10K measurements for each data-dependent section. Startup sections were estimated using their average measured time.

Code section	20K (s)	Code section	20K (s)
Imports	2.509	Report adata section	0.000
Settings	6.607	PCA	3.903
Argument parsing	0.001	Neighbors	67.982
Data loading	23.522	PAGA/commented section	0.000
Basic filtering	5.559	Louvain clustering	2.097
Normalization	1.423	UMAP	26.941
Highly variable genes	1.182	Write output	2.107
Scaling	1.721	Rank genes	49.156

The total estimated runtime is about **194.7 seconds (3.25 minutes)**. The largest predicted sections are Neighbors (68.0 s), Rank genes (49.2 s), UMAP (26.9 s), and data loading (23.5 s).

Performance bottlenecks

The main bottleneck changed as the dataset grew. UMAP was the slowest step for the 3K and 6K datasets, while Neighbors was the slowest step for the 10K dataset. Rank-gene testing also increased sharply at 10K. The cProfile results showed that pandas ranking was the largest cost inside rank_genes_groups, and sparse indexing became more important for PBMC10K.

For a 20K dataset, Neighbors and Rank genes are expected to take the most time. The estimates are approximate because only three dataset sizes were measured and several sections did not grow linearly from 6K to 10K.